



MODULE 09/09

Architecture Multi-tenant SaaS

Concevoir un SaaS qui scale

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre les 3 modèles de multi-tenancy et savoir lequel choisir
 - ✓ Implémenter l'isolation des données avec Row Level Security (RLS)
 - ✓ Créer un système d'onboarding tenant : inscription → provisionnement → app
 - ✓ Mettre en place l'audit log pour tracer les actions sensibles
 - ✓ Concevoir pour la scalabilité : indexes, soft delete, transactions
-

Planning de la semaine

JOUR 1	Modèles de multi-tenancy	2-3h
	• Modèle 1 : DB séparée par tenant (isolation maximale, coût élevé) • Modèle 2 : Schéma séparé par tenant (Postgres schema isolation) • Modèle 3 : tenant_id sur chaque table (choix de Social Seller) • Avantages/inconvénients de chaque modèle — quand choisir quoi	
JOUR 2	Isolation avec RLS	2-3h
	• Chaque table a tenant_id + RLS activé • current_tenant_id() : fonction SECURITY DEFINER qui lit auth.uid() • Service_role bypass RLS : uniquement pour le backend • Tester l'isolation : deux tenants ne voient pas les données de l'autre	
JOUR 3	Onboarding et provisionnement	2-3h
	• RPC SECURITY DEFINER : create_initial_tenant (contourne RLS pour créer les premières données) • Trigger on_tenant_created : automatiser la création du profil user • Flow : inscription → vérification email → création tenant → app • Gestion des plans (free/pro) et des limites	
JOUR 4	Audit log et soft delete	2-3h
	• Table audit_log : qui a fait quoi, quand, sur quelle ressource • Soft delete : deleted_at IS NULL au lieu de DELETE • Pourquoi : conformité RGPD, débogage, restauration possible • Logs structurés : pino/bunyan — JSON pour grep facile	
JOUR 5	Mini-projet de la semaine	3-4h
	• Concevoir un mini-SaaS 'Gestion de contacts' multi-tenant :	

- Schéma SQL : tenants, users, contacts (avec tenant_id partout)
- RLS politiques complètes sur chaque table
- RPC create_initial_tenant + trigger users
- Audit log sur création/suppression de contact
- Tester avec deux tenants : isolation parfaite

Concepts clés détaillés

Le modèle Social Seller : tenant_id partout

```
-- RÈGLE : tenant_id sur CHAQUE table métier
-- RÈGLE : RLS activé sur CHAQUE table
-- RÈGLE : Pas de DELETE → soft delete avec deleted_at

-- Schéma simplifié Social Seller
CREATE TABLE public.tenants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  plan text NOT NULL DEFAULT 'free',
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.conversations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES public.tenants(id), -- isolation
  platform text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

-- Même pattern sur : messages, orders, products, push_tokens...
```

RLS — la mécanique complète

```
-- Étape 1 : Fonction helper – résoudre le tenant depuis auth.uid()
CREATE OR REPLACE FUNCTION public.current_tenant_id()
RETURNS uuid LANGUAGE sql SECURITY DEFINER STABLE
SET search_path = public
AS $$ SELECT tenant_id FROM public.users WHERE id = auth.uid() $$;

-- Étape 2 : Activer RLS + créer les politiques
```

```

ALTER TABLE public.conversations ENABLE ROW LEVEL SECURITY;

CREATE POLICY conv_select ON public.conversations
FOR SELECT TO authenticated
USING (tenant_id = public.current_tenant_id());

CREATE POLICY conv_insert ON public.conversations
FOR INSERT TO authenticated
WITH CHECK (tenant_id = public.current_tenant_id());

-- social_connections : AUCUNE policy – backend service_role seulement
ALTER TABLE public.social_connections ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.social_connections FROM anon, authenticated;

```

Onboarding : RPC SECURITY DEFINER

```

-- Problème : à l'inscription, l'utilisateur n'a pas encore de tenant_id
-- donc RLS bloque tout INSERT. Solution : SECURITY DEFINER bypass.

CREATE OR REPLACE FUNCTION public.create_initial_tenant(
  p_tenant_name text,
  p_user_full_name text
)
RETURNS uuid LANGUAGE plpgsql SECURITY DEFINER
AS $$
DECLARE
  v_tenant_id uuid;
BEGIN
  -- 1. Créer le tenant
  INSERT INTO public.tenants(name) VALUES (p_tenant_name)
  RETURNING id INTO v_tenant_id;

  -- 2. Créer le user (lié à auth.uid())
  INSERT INTO public.users(id, tenant_id, role, full_name)
  VALUES (auth.uid(), v_tenant_id, 'merchant', p_user_full_name);

  RETURN v_tenant_id;
END;
$$;

-- Appel depuis le mobile :
const { data } = await supabase.rpc('create_initial_tenant', {

```

```
p_tenant_name: 'Ma Boutique',  
p_user_full_name: 'Mohamed Fomba'  
});
```

Audit log — traçabilité des actions

```
-- Table audit_log (migration 002 de Social Seller)  
  
CREATE TABLE public.audit_log (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id uuid NOT NULL REFERENCES public.tenants(id),  
  user_id uuid,  
  action text NOT NULL, -- 'order_status_changed', 'stock_adjusted'  
  table_name text NOT NULL,  
  record_id uuid,  
  old_value jsonb,  
  new_value jsonb,  
  created_at timestamptz NOT NULL DEFAULT now()  
);  
  
-- Utilisation dans services/auditLogService.ts  
async function recordAuditLog({ tenantId, userId, action, ... }) {  
  await supabaseAdmin.from('audit_log').insert({  
    tenant_id: tenantId,  
    user_id: userId,  
    action,  
    table_name: tableName,  
    record_id: recordId,  
    old_value: JSON.stringify(oldValue),  
    new_value: JSON.stringify(newValue),  
  });  
}
```

■ Le modèle Social Seller (*tenant_id shared*) est le plus simple à implémenter et suffisant pour démarrer. Quand un tenant dépasse 10 000 utilisateurs actifs, on peut envisager de migrer vers un schéma dédié. Pour les données ultra-sensibles (santé, finances), une DB séparée peut s'imposer.

Exercices pratiques

Exercice 1 — Schéma multi-tenant (45 min)

- Créer un schéma 'contacts_saas' avec : tenants, users, contacts
- Chaque contact a un tenant_id
- RLS : chaque user ne voit que les contacts de son tenant
- Insérer des données pour 2 tenants différents, vérifier l'isolation

Exercice 2 — Audit log automatique (1h)

- Créer un trigger qui insère dans audit_log après chaque UPDATE sur contacts
- Capturer old_value et new_value (OLD.* et NEW.* en plpgsql)
- Modifier un contact, vérifier qu'une ligne apparaît dans audit_log

Exercice 3 — Test d'isolation (30 min)

- Créer deux utilisateurs dans deux tenants différents
 - Se connecter avec l'utilisateur A, lister ses contacts
 - Tenter d'accéder à un contact du tenant B par son UUID direct
 - Vérifier que la réponse est vide (RLS bloque silencieusement)
-

Ressources pour aller plus loin

- Supabase RLS guide : supabase.com/docs/guides/database/postgres/row-level-security
- Multi-tenancy patterns : blog.lftechnology.com/2022/06/multi-tenant-saas-architecture
- 12-Factor App : 12factor.net — principes de design pour SaaS
- RGPD et données : cnil.fr/fr/rgpd-de-quoi-parle-t-on